

DevMerge

An Intelligent Collaborative Learning, Practice and
Academic Management Platform

Source Code Submission
(First and Last Portions of Source Code)

Prepared for Copyright Submission

Authors:
Navya Aggarwal
Shikha Tuteja

File: pages/admin/Dashboard.jsx

```
import React, { useEffect, useState } from 'react';
import { Link } from 'react-router-dom';
import { useAuthStore } from '../store/authStore';
import { motion } from 'framer-motion';
import adminService from '../services/api/adminService';
import UniversityDashboard from '../components/dashboard/UniversityDashboard';

export default function AdminDashboard() {
  const { user } = useAuthStore();
  const [stats, setStats] = useState(null);
  const [recentActivity, setRecentActivity] = useState([]);
  const [pendingApprovals, setPendingApprovals] = useState([]);
  const [systemHealth, setSystemHealth] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetchDashboardData();
  }, []);

  const fetchDashboardData = async () => {
    try {
      const [statsRes, healthRes, activityRes] = await Promise.allSettled([
        adminService.getSystemStats(),
        adminService.getSystemHealth(),
        adminService.getAuditLogs()
      ]);

      if (statsRes.status === 'fulfilled') {
        setStats(statsRes.value.data);
      }
      if (healthRes.status === 'fulfilled') {
        setSystemHealth(healthRes.value.data);
      }
      if (activityRes.status === 'fulfilled') {
        const activities = activityRes.value.data || activityRes.value || [];
        setRecentActivity(Array.isArray(activities) ? activities.slice(0, 10).map((log, idx) => ({
          id: log._id || idx,
          type: log.action || 'activity',

```

```

        action: log.description || log.action || 'System Update',
        timestamp: log.createdAt || new Date(),
        user: log.userId?.name || log.userId || 'System',
        icon: getActivityIcon(log.action)
      ))) : []);
    }

    } catch (error) {
      console.error('Error fetching admin dashboard data:', error);
    } finally {
      setLoading(false);
    }
  };

const getActivityIcon = (action) => {
  const iconMap = {
    'create': '✚',
    'update': '✎',
    'delete': '🗑',
    'login': '🔑',
    'register': '👤',
    'submit': '📝',
    'grade': '📊',
    'approve': '✅',
    'reject': '❌'
  };
  const actionLower = (action || '').toLowerCase();
  for (const [key, icon] of Object.entries(iconMap)) {
    if (actionLower.includes(key)) return icon;
  }
  return '📌';
};

if (loading) {
  return (
    <div className="min-h-screen bg-slate-50 dark:bg-dark-950 flex items-center justify-center">
      <div className="text-center">
        <div className="animate-spin rounded-full h-16 w-16 border-4 border-indigo-200 border-t-indigo-600 mx-auto mb-4 shadow-xl"></div>

```

```

        <p className="text-indigo-600 font-black uppercase tracking-[0.4em]
text-[10px]">Accessing Admin Nexus Cluster...</p>
    </div>
</div>
);
}

return (
    <div className="space-y-8">
        { /* University Dashboard Component */ }
        <UniversityDashboard />

        { /* Admin-Specific Content */ }
        <motion.div
            initial={{ opacity: 0 }}
            animate={{ opacity: 1 }}
            className="max-w-[1700px] mx-auto font-sans"
        >
            <div className="flex flex-col xl:flex-row justify-between items-start xl:items-center mb-12
lg:mb-20 gap-8">
                <div className="w-full xl:w-auto">
                    <h1 className="text-4xl sm:text-5xl md:text-6xl lg:text-7xl font-black text-slate-900
dark:text-white tracking-tighter uppercase leading-none italic">
                        Admin <span className="text-indigo-600">Nexus</span>
                    </h1>
                    <p className="text-slate-400 font-bold uppercase tracking-[0.4em] text-[9px]
md:text-[10px] mt-4 flex items-center gap-2">
                        <span className="w-2.5 h-2.5 bg-indigo-500 rounded-full animate-pulse
shadow-[0_0_10px_rgba(79,70,229,0.5)]"></span>
                        Infrastructure & System Oversight · Operational Clear
                    </p>
                </div>
                <div className="w-full xl:w-auto bg-white dark:bg-dark-900 px-6 md:px-10 py-6
rounded-[2.5rem] md:rounded-[3.5rem] border border-slate-100 dark:border-dark-800
shadow-3xl flex items-center gap-4 md:gap-8 relative overflow-hidden group">
                    <div className="absolute inset-0 bg-gradient-to-br from-indigo-600/5
to-transparent"></div>
                    <div className="w-14 h-14 md:w-16 md:h-16 bg-indigo-600 rounded-2xl
md:rounded-[1.5rem] flex items-center justify-center text-white font-black text-xl italic
shadow-2xl relative z-10 group-hover:rotate-12 transition-transform shrink-0">

```

```

        {user.name?.charAt(0) || 'A'}
      </div>
      <div className="relative z-10 min-w-0">
        <p className="text-[10px] font-black text-slate-400 uppercase tracking-widest
leading-none mb-1">Administrative Node</p>
        <p className="text-xl md:text-2xl font-black text-slate-900 dark:text-white
leading-none whitespace-nowrap italic truncate">{user.name}</p>
      </div>
    </div>
  </div>
</div>

{/* Stats Grid */}
<div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-6 md:gap-8 lg:gap-10
mb-12 lg:mb-20">
  {[
    { label: 'Total Users', value: stats?.totalUsers || 0, trend: `+${stats?.newUsersToday || 0}`,
color: 'blue', icon: '👥' },
    { label: 'Active Courses', value: stats?.activeCourses || 0, trend: `of ${stats?.courses || 0}`,
color: 'emerald', icon: '🎓' },
    { label: 'Pending Reviews', value: stats?.pendingSubmissions || 0, trend: 'Audit Queue',
color: 'amber', icon: '⏳' },
    { label: 'Average Grade', value: `${stats?.avgGrade || 0}%`, trend: 'System Avg', color:
'purple', icon: '📊' },
  ].map((stat, i) => (
    <motion.div
      key={i}
      whileHover={{ y: -10, scale: 1.02 }}
      className={`bg-white dark:bg-dark-900 p-8 md:p-10 rounded-[3rem]
md:rounded-[3.5rem] shadow-2xl border-b-[8px] md:border-b-[12px] border-${stat.color}-500
group relative overflow-hidden transition-all duration-300`}
    >
      <div className="absolute top-0 right-0 p-8 text-6xl md:text-8xl opacity-[0.03]
group-hover:opacity-10 transition-opacity pointer-events-none group-hover:rotate-12
group-hover:scale-125">{stat.icon}</div>
      <div className="relative z-10">
        <p className={`text-[10px] font-black text-${stat.color}-600
dark:text-${stat.color}-400 uppercase tracking-widest mb-4 leading-none`}>{stat.label}</p>
        <div className="flex items-baseline gap-2">
          <p className="text-4xl md:text-5xl lg:text-6xl font-black text-slate-900
dark:text-white tracking-tighter leading-none italic">{stat.value}</p>

```

```
      <p className="text-[9px] md:text-[10px] font-black text-slate-400 uppercase tracking-widest opacity-60 italic">{stat.trend}</p>
```

```
    </div>
```

```
  </div>
```

```
</motion.div>
```

```
  )})
```

```
</div>
```

```
{/* Quick Actions */}
```

```
<div className="bg-white dark:bg-dark-900 rounded-[3rem] md:rounded-[4.5rem] p-8 md:p-14 shadow-3xl border border-slate-50 dark:border-dark-800 mb-12 lg:mb-20 relative overflow-hidden group">
```

```
  <div className="absolute top-0 right-0 p-12 opacity-[0.02] text-9xl font-black italic pointer-events-none select-none">CONTROLS</div>
```

```
  <h2 className="text-xl md:text-2xl font-black mb-10 md:mb-14 dark:text-white uppercase tracking-tighter flex items-center gap-4 italic relative z-10">
```

```
    <span>🏠</span> Macro Management Hub
```

```
  </h2>
```

```
  <div className="grid grid-cols-2 sm:grid-cols-3 md:grid-cols-4 lg:grid-cols-6 gap-4 md:gap-8 relative z-10">
```

```
    {[
```

```
      { name: 'Departments', path: '/admin/departments', icon: '🏢', color: 'blue' },
```

```
      { name: 'Programs', path: '/admin/programs', icon: '📅', color: 'indigo' },
```

```
      { name: 'Academic Years', path: '/admin/academic-years', icon: '🕒', color: 'amber' },
```

```
      { name: 'Semesters', path: '/admin/semesters', icon: '📅', color: 'emerald' },
```

```
      { name: 'Sections', path: '/admin/sections', icon: '🏠', color: 'violet' },
```

```
      { name: 'Enrollments', path: '/admin/enrollments', icon: '📖', color: 'rose' },
```

```
    ].map((action, i) => (
```

```
      <Link key={i} to={action.path} className="flex flex-col items-center group/action">
```

```
        <div className={`w-14 h-14 md:w-20 md:h-20 rounded-[1.5rem]
```

```
md:rounded-[2.5rem] bg-${action.color}-50 dark:bg-${action.color}-900/20 border-2
```

```
border-${action.color}-100 dark:border-${action.color}-800/30 flex items-center justify-center
```

```
text-2xl md:text-4xl shadow-md group-hover/action:bg-${action.color}-600
```

```
group-hover/action:text-white transition-all duration-300 transform
```

```
group-hover/action:-translate-y-2`}>
```

```
          {action.icon}
```

```
        </div>
```

```
        <span className="mt-4 text-[9px] md:text-[10px] font-black text-slate-500 dark:text-slate-400 uppercase tracking-widest text-center italic">{action.name}</span>
```

```
      </Link>
```

```

    ))}
  </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-3 gap-8 lg:gap-12">
  { /* Recent Activity */ }
  <div className="lg:col-span-2 bg-white dark:bg-dark-900 rounded-[3rem] p-8 md:p-12
shadow-3xl border border-slate-50 dark:border-dark-800">
    <div className="flex flex-col sm:flex-row items-start sm:items-center justify-between
mb-10 md:mb-12 gap-4">
      <h2 className="text-xl md:text-2xl font-black text-slate-900 dark:text-white uppercase
tracking-tighter italic">System Audit Log</h2>
      <Link to="/admin/audit-logs" className="text-[10px] font-black text-indigo-600
dark:text-indigo-400 uppercase tracking-[0.3em] hover:translate-x-2 transition-transform italic">
        Archives →
      </Link>
    </div>
    <div className="space-y-4 md:space-y-6">
      {recentActivity.length > 0 ? recentActivity.map((activity, idx) => (
        <motion.div
          key={activity.id}
          initial={{ opacity: 0, x: -10 }}
          animate={{ opacity: 1, x: 0 }}
          transition={{ delay: idx * 0.05 }}
          className="flex items-center gap-4 md:gap-8 p-6 md:p-8 bg-slate-50
dark:bg-dark-950/50 rounded-[2rem] md:rounded-[2.5rem] border border-slate-100
dark:border-dark-800 hover:shadow-2xl transition-all group"
        >
          <div className="text-2xl md:text-3xl p-4 bg-white dark:bg-dark-900 rounded-2xl
shadow-lg border border-slate-100 dark:border-dark-800 group-hover:rotate-12
transition-transform shrink-0">{activity.icon}</div>
          <div className="flex-1 min-w-0">
            <p className="text-sm md:text-base font-black text-slate-900 dark:text-white
uppercase tracking-tighter leading-none mb-2 italic truncate">{activity.action}</p>
            <p className="text-[9px] md:text-[10px] font-black text-slate-400 uppercase
tracking-widest leading-none italic">{activity.user}</p>
          </div>
        </motion.div>
      )) : (

```

```

    <div className="py-20 text-center bg-slate-50 dark:bg-dark-950/50 rounded-[2.5rem]
border-4 border-dashed border-slate-100 dark:border-dark-800">
      <p className="text-[10px] font-black text-slate-300 uppercase tracking-[0.4em] italic
leading-loose">Matrix Stream Idle...</p>
    </div>
  )}
</div>
</div>

```

```

<div className="space-y-8 lg:space-y-12">
  { /* Pending Approvals */ }
  <div className="bg-white dark:bg-dark-900 rounded-[3rem] p-8 md:p-12 shadow-3xl
border-l-[12px] md:border-l-[20px] border-amber-500 overflow-hidden relative group">
    <div className="absolute top-0 right-0 w-32 h-32 bg-amber-500/10 rounded-full -mr-16
-mt-16 blur-3xl"></div>
    <div className="flex items-center justify-between mb-10 md:mb-12 relative z-10">
      <h2 className="text-lg md:text-xl font-black text-slate-900 dark:text-white uppercase
tracking-tighter italic">Process Queue</h2>
      <span className="bg-amber-100 dark:bg-amber-900/40 text-amber-600
dark:text-amber-400 text-[9px] font-black px-4 py-1.5 rounded-full uppercase tracking-widest
border border-amber-500/20">
        {pendingApprovals.length} Blocked
      </span>
    </div>
    <div className="space-y-6 relative z-10">
      {pendingApprovals.length > 0 ? pendingApprovals.map((item) => (
        <div key={item.id} className="p-6 md:p-8 bg-amber-50 dark:bg-dark-950/50
rounded-[2rem] border border-amber-100 dark:border-dark-800 transition-all hover:bg-white
dark:hover:bg-dark-900">
          <div className="flex items-start justify-between mb-6">
            <div>
              <p className="text-[8px] md:text-[9px] font-black text-amber-600 uppercase
tracking-[0.2em] mb-2">{item.type} Node Submission</p>
              <p className="text-sm md:text-base font-black text-slate-900 dark:text-white
uppercase tracking-tighter italic leading-tight">{item.title}</p>
              <p className="text-[9px] font-bold text-slate-400 uppercase tracking-widest
mt-2">Source: {item.requester}</p>
            </div>
          </div>
        </div>
      )}
    </div>
  </div>

```



```

        <button className="bg-emerald-600 text-white text-[9px] font-black py-4
rounded-xl md:rounded-2xl hover:bg-slate-900 transition-all uppercase tracking-widest italic
shadow-xl shadow-emerald-600/20 hover:shadow-none">
            Authorize
        </button>
        <button className="bg-slate-200 dark:bg-dark-800 text-slate-800 dark:text-white
text-[9px] font-black py-4 rounded-xl md:rounded-2xl hover:bg-rose-600 hover:text-white
transition-all uppercase tracking-widest italic">
            Dismiss
        </button>
    </div>
</div>
)) : (
    <div className="py-16 text-center bg-slate-50 dark:bg-dark-950/50 rounded-[2.5rem]
border-2 border-dashed border-slate-100 dark:border-dark-800">
        <p className="text-[9px] font-black text-slate-400 uppercase tracking-[0.3em]
italic">Queue Optimized</p>
    </div>
    </div>
</div>

{ /* System Health */ }
<div className="bg-slate-900 rounded-[3rem] md:rounded-[4rem] p-8 md:p-12
text-white shadow-3xl relative overflow-hidden group border-t-[12px] md:border-t-[20px]
border-emerald-600">
    <div className="absolute top-0 right-0 w-64 h-64 bg-emerald-600/10 rounded-full
-mr-32 -mt-32 blur-[100px]"></div>
    <h2 className="text-lg md:text-xl font-black uppercase tracking-tighter mb-10
md:mb-12 italic flex items-center justify-between relative z-10 leading-none">
        <span>Infrastructure Vitality</span>
        <span className="w-3 h-3 bg-emerald-500 rounded-full animate-pulse
shadow-[0_0_10px_rgba(16,185,129,0.5)]"></span>
    </h2>
    <div className="space-y-8 md:space-y-10 relative z-10">
        {[
            { label: 'CPU Cluster Load', value: systemHealth?.cpu || 14, color: 'blue' },
            { label: 'Memory Allocation', value: systemHealth?.memory || 42, color: 'emerald' },
            { label: 'Storage Footprint', value: systemHealth?.disk || 68, color: 'amber' },
        ].map((vital, i) => (

```

```

    <div key={i}>
      <div className="flex items-center justify-between mb-3 text-[9px] md:text-[10px]
font-black uppercase tracking-widest text-slate-400">
        <span>{vital.label}</span>
        <span className={`text-${vital.color}-400 italic
font-mono`} >{vital.value}%</span>
      </div>
      <div className="h-3 md:h-4 w-full bg-white/5 rounded-full overflow-hidden p-0.5
md:p-1 border border-white/5 shadow-inner">
        <motion.div
          initial={{ width: 0 }}
          animate={{ width: `${vital.value}%` }}
          transition={{ duration: 1, delay: i * 0.2 }}
          className={`h-full rounded-full bg-gradient-to-r shadow-lg ${vital.value < 40 ?
'from-blue-500 to-indigo-600' :
          vital.value < 70 ? 'from-emerald-500 to-teal-600' :
            'from-amber-500 to-rose-600'
          }}
        />
      </div>
    </div>
  )}
</div>

<div className="grid grid-cols-3 gap-3 mt-12 md:mt-16 relative z-10">
  {[ 'Database', 'Cache', 'API' ].map((service, i) => (
    <div key={i} className="text-center p-3 md:p-4 bg-white/5 rounded-xl
md:rounded-2xl border border-white/5 backdrop-blur-md transition-all hover:bg-white/10">
      <div className="w-1.5 h-1.5 md:w-2 md:h-2 bg-emerald-500 rounded-full mx-auto
mb-2 shadow-[0_0_10px_rgba(16,185,129,0.5)]"></div>
      <p className="text-[8px] md:text-[9px] font-black text-slate-400 uppercase
tracking-tighter leading-none mb-1">{service}</p>
      <p className="text-[9px] md:text-[10px] font-black text-emerald-400 uppercase
tracking-widest italic leading-none">OK</p>
    </div>
  )}
</div>
</div>
</div>
</div>

```

```

    </motion.div>
  </div>
);
}

```

File: services/api/[hodService.js](#)

```

import apiClient from './apiClient';
import { ENDPOINTS } from '../config/urls';

/**
 * HOD (Head of Department) Management Service
 */
const normalizeDepartmentId = (deptRef) => {
  if (!deptRef) return '';
  if (typeof deptRef === 'string') return deptRef;
  return deptRef._id || deptRef.id || '';
};

const toError = (error, fallbackMessage) => {
  const message = error?.formattedMessage || error?.response?.data?.message || error?.message ||
  fallbackMessage;
  return new Error(message);
};

const hodService = {
  // Get department overview including critical rates and totals
  getDepartmentOverview: async (deptRef) => {
    try {
      const deptId = normalizeDepartmentId(deptRef);
      if (!deptId) return { success: true, data: {} };
      const response = await
apiClient.get(ENDPOINTS.INSTITUTIONAL.DEPARTMENT_OVERVIEW(deptId));
      return response.data;
    } catch (error) {
      throw toError(error, 'Failed to load department overview');
    }
  },

  // Get cognitive load indicators for department students
  getCognitiveLoadData: async (deptRef) => {

```

```

    try {
      const deptId = normalizeDepartmentId(deptRef);
      if (!deptId) return { success: true, data: [] };
      const response = await
apiClient.get(ENDPOINTS.INSTITUTIONAL.COG_LOAD(deptId));
      return response.data;
    } catch (error) {
      throw toError(error, 'Failed to load cognitive load data');
    }
  },

  // Get operational efficiency metrics for department
  getEfficiencyMetrics: async (deptRef) => {
    try {
      const deptId = normalizeDepartmentId(deptRef);
      if (!deptId) return { success: true, data: {} };
      const response = await
apiClient.get(ENDPOINTS.INSTITUTIONAL.EFFICIENCY(deptId));
      return response.data;
    } catch (error) {
      throw toError(error, 'Failed to load efficiency metrics');
    }
  },

  // Get accreditation and attainment report for department
  getAccreditationData: async (deptRef) => {
    try {
      const deptId = normalizeDepartmentId(deptRef);
      if (!deptId) return { success: true, data: {} };
      const response = await
apiClient.get(ENDPOINTS.INSTITUTIONAL.ACCREDITATION(deptId));
      return response.data;
    } catch (error) {
      throw toError(error, 'Failed to load accreditation data');
    }
  },

  // Get faculty performance rankings for department
  getFacultyPerformance: async (deptRef) => {
    try {

```

```

        const deptId = normalizeDepartmentId(deptRef);
        if (!deptId) return { success: true, data: [] };
        const response = await
apiClient.get(ENDPOINTS.INSTITUTIONAL.FACULTY_PERFORMANCE(deptId));
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to load faculty performance');
    }
},

// Get departmental courses with faculty mappings
getDepartmentCourses: async (deptRef) => {
    try {
        const deptId = normalizeDepartmentId(deptRef);
        if (!deptId) return { success: true, data: [] };
        const response = await
apiClient.get(ENDPOINTS.INSTITUTIONAL.DEPARTMENT_COURSES(deptId));
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to load department courses');
    }
},

// Get workload comparison across all faculty in a department
getWorkloadDistribution: async (deptRef) => {
    try {
        const deptId = normalizeDepartmentId(deptRef);
        if (!deptId) return { success: true, data: { workloadDistribution: [] } };
        const response = await
apiClient.get(ENDPOINTS.INSTITUTIONAL.WORKLOAD_DISTRIBUTION(deptId));
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to load workload distribution');
    }
},

// Department Settings Management
getDepartmentSettings: async (departmentId) => {
    try {
        if (!departmentId) throw new Error('Department ID is required');

```

```

        const response = await apiClient.get(`/api/hod/department-settings/${departmentId}`);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to load department settings');
    }
},

updateDepartmentSettings: async (departmentId, settings) => {
    try {
        if (!departmentId) throw new Error('Department ID is required');
        const response = await apiClient.put(`/api/hod/department-settings/${departmentId}`,
settings);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to update department settings');
    }
},

// Course Management
getCourses: async (filters = {}) => {
    try {
        const queryParams = new URLSearchParams(filters).toString();
        const response = await apiClient.get(`/api/hod/courses?${queryParams}`);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to load courses');
    }
},

createCourse: async (courseData) => {
    try {
        const response = await apiClient.post('/api/hod/courses', courseData);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to create course');
    }
},

updateCourse: async (courseId, courseData) => {
    try {

```

```

        if (!courseId) throw new Error('Course ID is required');
        const response = await apiClient.put(`/api/hod/courses/${courseId}`, courseData);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to update course');
    }
},

```

```

deleteCourse: async (courseId) => {
    try {
        if (!courseId) throw new Error('Course ID is required');
        const response = await apiClient.delete(`/api/hod/courses/${courseId}`);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to delete course');
    }
},

```

// Program Management

```

getPrograms: async (filters = {}) => {
    try {
        const queryParams = new URLSearchParams(filters).toString();
        const response = await apiClient.get(`/api/hod/programs?${queryParams}`);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to load programs');
    }
},

```

```

createProgram: async (programData) => {
    try {
        const response = await apiClient.post('/api/hod/programs', programData);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to create program');
    }
},

```

```

updateProgram: async (programId, programData) => {
    try {

```

```

        if (!programId) throw new Error('Program ID is required');
        const response = await apiClient.put(`/api/hod/programs/${programId}`, programData);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to update program');
    }
},

```

```

deleteProgram: async (programId) => {
    try {
        if (!programId) throw new Error('Program ID is required');
        const response = await apiClient.delete(`/api/hod/programs/${programId}`);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to delete program');
    }
},

```

// Department Management

```

getDepartments: async (filters = {}) => {
    try {
        const queryParams = new URLSearchParams(filters).toString();
        const response = await apiClient.get(`/api/hod/departments?${queryParams}`);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to load departments');
    }
},

```

```

createDepartment: async (departmentData) => {
    try {
        const response = await apiClient.post('/api/hod/departments', departmentData);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to create department');
    }
},

```

```

updateDepartment: async (departmentId, departmentData) => {
    try {

```



```

        if (!departmentId) throw new Error('Department ID is required');
        const response = await apiClient.put(`/api/hod/departments/${departmentId}`,
departmentData);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to update department');
    }
},

```

```

deleteDepartment: async (departmentId) => {
    try {
        if (!departmentId) throw new Error('Department ID is required');
        const response = await apiClient.delete(`/api/hod/departments/${departmentId}`);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to delete department');
    }
},

```

// Section Management

```

getSections: async (filters = {}) => {
    try {
        const queryParams = new URLSearchParams(filters).toString();
        const response = await apiClient.get(`/api/hod/sections?${queryParams}`);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to load sections');
    }
},

```

```

createSection: async (sectionData) => {
    try {
        const response = await apiClient.post('/api/hod/sections', sectionData);
        return response.data;
    } catch (error) {
        throw toError(error, 'Failed to create section');
    }
},

```

```

updateSection: async (sectionId, sectionData) => {

```

```

    try {
      if (!sectionId) throw new Error('Section ID is required');
      const response = await apiClient.put(`/api/hod/sections/${sectionId}`, sectionData);
      return response.data;
    } catch (error) {
      throw toError(error, 'Failed to update section');
    }
  },

  deleteSection: async (sectionId) => {
    try {
      if (!sectionId) throw new Error('Section ID is required');
      const response = await apiClient.delete(`/api/hod/sections/${sectionId}`);
      return response.data;
    } catch (error) {
      throw toError(error, 'Failed to delete section');
    }
  },

  // Auto-assign students to sections
  autoAssignStudents: async (semesterId, config = {}) => {
    try {
      if (!semesterId) throw new Error('Semester ID is required');
      const response = await apiClient.post('/api/hod/sections/auto-assign', { semesterId, config
    });
    return response.data;
    } catch (error) {
      throw toError(error, 'Failed to auto-assign students');
    }
  },

  // Faculty Assignment
  assignFaculty: async (assignmentData) => {
    try {
      const response = await apiClient.post('/api/hod/faculty-assignment', assignmentData);
      return response.data;
    } catch (error) {
      throw toError(error, 'Failed to assign faculty');
    }
  },

```

```
// Course Approvals
getCourseApprovals: async (filters = {}) => {
  try {
    const queryParams = new URLSearchParams(filters).toString();
    const response = await apiClient.get(`/api/hod/course-approvals?${queryParams}`);
    return response.data;
  } catch (error) {
    throw toError(error, 'Failed to load course approvals');
  }
},
```

```
// Audit Logs
getAuditLogs: async (filters = {}) => {
  try {
    const queryParams = new URLSearchParams(filters).toString();
    const response = await apiClient.get(`/api/hod/audit-logs?${queryParams}`);
    return response.data;
  } catch (error) {
    throw toError(error, 'Failed to load audit logs');
  }
},
```

```
getAuditStatistics: async (filters = {}) => {
  try {
    const queryParams = new URLSearchParams(filters).toString();
    const response = await apiClient.get(`/api/hod/audit-statistics?${queryParams}`);
    return response.data;
  } catch (error) {
    throw toError(error, 'Failed to load audit statistics');
  }
},
```

```
// Broadcast System
getBroadcastStats: async () => {
  try {
    const response = await apiClient.get('/api/hod/broadcast-stats');
    return response.data;
  } catch (error) {
    throw toError(error, 'Failed to load broadcast statistics');
  }
}
```

```

    }
  },

  sendBroadcast: async (broadcastData) => {
    try {
      const response = await apiClient.post('/api/hod/broadcast', broadcastData);
      return response.data;
    } catch (error) {
      throw toError(error, 'Failed to send broadcast');
    }
  },

  getBroadcastHistory: async (filters = {}) => {
    try {
      const queryParams = new URLSearchParams(filters).toString();
      const response = await apiClient.get(`/api/hod/broadcast-history?${queryParams}`);
      return response.data;
    } catch (error) {
      throw toError(error, 'Failed to load broadcast history');
    }
  },

  // Gamification
  getGamificationConfig: async () => {
    try {
      const response = await apiClient.get('/api/hod/gamification-config');
      return response.data;
    } catch (error) {
      throw toError(error, 'Failed to load gamification configuration');
    }
  },

  updateGamificationMultiplier: async (multiplierData) => {
    try {
      const response = await apiClient.post('/api/hod/gamification-multiplier', multiplierData);
      return response.data;
    } catch (error) {
      throw toError(error, 'Failed to update gamification multiplier');
    }
  },

```

```

// Department Stats (existing)
getDepartmentStats: async (departmentId) => {
  try {
    if (!departmentId) throw new Error('Department ID is required');
    const response = await apiClient.get('/api/hod/department-stats');
    return response.data;
  } catch (error) {
    throw toError(error, 'Failed to load department statistics');
  }
},
// Faculty Load (existing)
getFacultyLoad: async (departmentId) => {
  try {
    if (!departmentId) throw new Error('Department ID is required');
    const response = await apiClient.get('/api/hod/faculty-load');
    return response.data;
  } catch (error) {
    throw toError(error, 'Failed to load faculty load');
  }
}
};

export default hodService;

```